

Operații aritmetice

1 Obiective

Lucrarea de față îl familiarizează pe student cu principalele recurențe matematice și cu tipurile de recursivitate.

2 Considerații teoretice

2.1 Operatorul „is”

În Prolog expresiile matematice sunt evaluate folosind operatorul „is”. Expresia matematică se scrie în dreapta la *is* și nu poate să conțină variabile care sunt neinițializate.

Exemple:

```
?- X is sqrt(4).  
X = 2.0.
```

```
?- Y = 10, X is Y mod 2.  
X = 0.
```

```
?- X is (1+2*3)/7-1.  
X = 0.
```

2.2 Cel mai mare divizor comun (CMMDC)

Pentru a calcula cel mai mare divizor comun vom folosi algoritmul lui Euclid.
Pseudocod varianta 1 (pentru numere pozitive):

```
cmmdc(a,a) = a  
cmmdc(a,b) = cmmdc(a-b, b), dacă a>b  
cmmdc(a,b) = cmmdc(a, b-a), dacă b>a
```

Pseudocod varianta 2 (mai eficientă pentru numere mari, numere nenule):

```
cmmdc(a,0) = a  
cmmdc(a,b) = cmmdc(b, a mod b)
```

Deoarece în Prolog predicatele returnează doar adevărat/fals (yes/no), atunci va trebui să adăugăm în lista de parametri ai predicatului și rezultatul.

% Varianta 1

```
cmmdc1(X,X,X). % parametrul 3 îi rezultatul la cmmdc  
cmmdc1(X,Y,Z) :- X>Y, Diff is X-Y, cmmdc1(Diff,Y,Z).  
cmmdc1(X,Y,Z) :- X<Y, Diff is Y-X, cmmdc1(X,Diff,Z).
```

% Varianta 2

```
cmmdc2(X,0,X). % parametrul 3 îi rezultatul la cmmdc
cmmdc2(X,Y,Z) :- Rest is X mod Y, cmmdc2(Y,Rest,Z).
```

Exemplu de urmărire a execuției (cu trace):

```
[trace] 3 ?- cmmdc1(3,3,R).
  Call: (7) cmmdc1(3, 3, _G1614) ?    % se unifică cu prima clauză
  Exit: (7) cmmdc1(3, 3, 3) ?        % iese cu succes
R = 3 ;                               % repetăm întrebarea
  Redo: (7) cmmdc1(3, 3, _G1614) ?    % se unifică cu clauza 2
  Call: (8) 3>3 ?                    % primul apel din clauza 2
  Fail: (8) 3>3 ?                    % eșuează și trece la
următoarea clauză
  Redo: (7) cmmdc1(3, 3, _G1614) ?    % se unifică cu clauza 3
  Call: (8) 3<3 ?                    % primul apel din clauza 3
  Fail: (8) 3<3 ?                    % eșuează
  Fail: (7) cmmdc1(3, 3, _G1614) ?    % eșuează (nu mai există altă
clauză cu care să se unifice)
false.
```

```
[trace] 6 ?- cmmdc1(3,5,R).
  Call: (7) cmmdc1(3, 5, _G1614) ?    % se unifică cu clauza 2
  Call: (8) 3>5 ?                    % primul apel din clauza 2
  Fail: (8) 3>5 ?                    % eșuează și trece la
următoarea clauză
  Redo: (7) cmmdc1(3, 5, _G1614) ?    % se unifică cu clauza 3
  Call: (8) 3<5 ?                    % primul apel din clauza 3
  Exit: (8) 3<5 ?                    % succes
  Call: (8) _G1693 is 5-3 ?            % al doilea apel din clauza 3
  Exit: (8) 2 is 5-3 ?                % succes
  Call: (8) cmmdc1(3, 2, _G1614) ?    % apelul recursiv din clauza 3
  Call: (9) 3>2 ?                    % primul apel din clauza 2
  Exit: (9) 3>2 ?                    % succes
  Call: (9) _G1696 is 3-2 ?            % al doilea apel din clauza 2
  Exit: (9) 1 is 3-2 ?                % succes
  Call: (9) cmmdc1(1, 2, _G1614) ?    % apelul recursiv din clauza 3
  Call: (10) 1>2 ?                    % etc.
  Fail: (10) 1>2 ?
  Redo: (9) cmmdc1(1, 2, _G1614) ?
  Call: (10) 1<2 ?
  Exit: (10) 1<2 ?
  Call: (10) _G1699 is 2-1 ?
  Exit: (10) 1 is 2-1 ?
  Call: (10) cmmdc1(1, 1, _G1614) ?
  Exit: (10) cmmdc1(1, 1, 1) ?
  Exit: (9) cmmdc1(1, 2, 1) ?
  Exit: (8) cmmdc1(3, 2, 1) ?
  Exit: (7) cmmdc1(3, 5, 1) ?
R = 1
```

Urmărește execuția la:

?- cmmdc1(30,24,X).

?- cmmdc1(15,2,X).

?- cmmdc1(4,1,X).

2.3 Factorial

Recurența matematică pentru factorial este:

$\text{factorial}(0) = 1$

$\text{factorial}(n) = n * \text{factorial}(n-1)$, pentru $n > 0$

2.3.1 Recursivitatea înapoi (backwards)

În cadrul recursivității înapoi, rezultatul este construit la întoarcerea din recursivitate. Ceea ce înseamnă că inițializarea rezultatului se face după apelul recursiv.

$\text{fact1}(0,1).$

$\text{fact1}(N,F) :- N1 \text{ is } N-1, \text{fact1}(N1,F1), F \text{ is } N * F1.$

Urmărește execuția la:

?- $\text{fact1}(6, 720).$

?- $N=6, \text{fact1}(N, 120).$

?- $\text{fact1}(6, F).$

?- $\text{fact1}(N,720).$

?- $\text{fact1}(N,F).$

Dacă se repetă întrebarea, va trebui să ne asigurăm că N nu ajunge să fie negativ.

Clauza a doua din predicat ar trebui să fie:

$\text{fact1}(N,F) :- N > 0, N1 \text{ is } N-1, \text{fact1}(N1,F1), F \text{ is } N * F1.$

2.3.2 Recursivitate înainte (forward)

În cadrul recursivității înainte, rezultatul este construit într-un acumulator. Valoarea acumulatorului trebuie asignată rezultatului în ultimul pas de recursivitate (clauza de terminare). Valoarea noului acumulator se va calcula înainte de apelul recursiv.

$\text{fact2}(0,Acc,F) :- F = Acc.$

$\text{fact2}(N,Acc,F) :- N > 0, N1 \text{ is } N-1, Acc1 \text{ is } Acc * N, \text{fact2}(N1,Acc1,F).$

$\text{fact2}(N,F) :- \text{fact2}(N,1,F).$ % acumulatorul este inițializat cu 1

3 Exerciții

1. Scrieți un predicat care să calculeze cel mai mic multiplu comun (CMMMC). (Sugestie: cmmmc între două numere naturale este raportul dintre produsul lor și cmmdc)
2. Scrieți predicatul pentru ridicarea la putere în cele două tipuri de recursivitate.
3. Scrieți predicatul care calculează al n -lea număr din șirul lui Fibonacci. Formula de recurență este:

$$\text{fib}(0) = 0$$

$$\text{fib}(1) = 1$$

$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2), \text{ pentru } n > 0$$

4. (*) Scrieți predicatul anterior folosind doar un singur apel recursiv.
5. Scrieți predicatul *triangle/3* care verifică dacă cei trei parametri pot reprezenta lungimile laturilor unui triunghi. Suma oricăror două laturi trebuie să fie mai mare decât a treia latură.
6. Scrieți un predicat (*solve/4*) care să rezolve ecuația de gradul doi ($a \cdot x^2 + b \cdot x + c = 0$). Predicatul trebuie să aibă 3 parametri de intrare (A, B, C) și un parametru de ieșire (X). La repetarea întrebării trebuie să se obțină a doua soluție (distinctă) dacă există.